Big Data Assignment 2 Report

Name: Polina Kuzminykh
Email: p.kuzminykh@innopolis.university
Group: DS-02

Methodology

MapReduce pipeline:

The mapper tokenizes text using a simple alphanumeric regular expression. It emits four logical record types: DOC_LEN, DOC_META, TERM_TF, and TERM_DF. DOC_LEN stores the document length required by BM25. DOC_META preserves the title for result display. TERM_TF stores term frequency per document. TERM_DF marks the presence of a term in a document and is later aggregated to a document frequency.

The reducer merges all mapper outputs and computes the final global counts. It re-emits compact records that are easy to load into Cassandra: document lengths, document titles, term document frequencies, postings, and corpus statistics such as DOC_COUNT and AVGDL.

create_index.sh

Firstly it removes previous outputs so that repeated runs do not fail because of existing HDFS directories. Then it finds the Hadoop streaming jar dynamically, which avoids hard-coding a single Hadoop installation path. The output is written to /indexer, which is later consumed by the Cassandra loading step.

store_index.py

Instead of leaving the index in HDFS only, the script loads it into Cassandra so that the ranking phase can query a structured datastore quickly. The script creates the keyspace and tables if they do not already exist. Separate tables are used for document lengths, document metadata, term document frequencies, term postings, and global stats. This makes the query-time code simpler and improves maintainability. Batch inserts are used to reduce network overhead. The code reads the HDFS output once, parses each record type, and stores the data with the Cassandra Python driver.

query.py

Firstly the query string is normalized and tokenized into separate terms. Then the script retrieves the required term statistics and document metadata from Cassandra. Using these values, it computes BM25 scores for documents that contain the query terms. After scoring, it sorts the documents by descending relevance score and keeps top 10 results.

search.sh

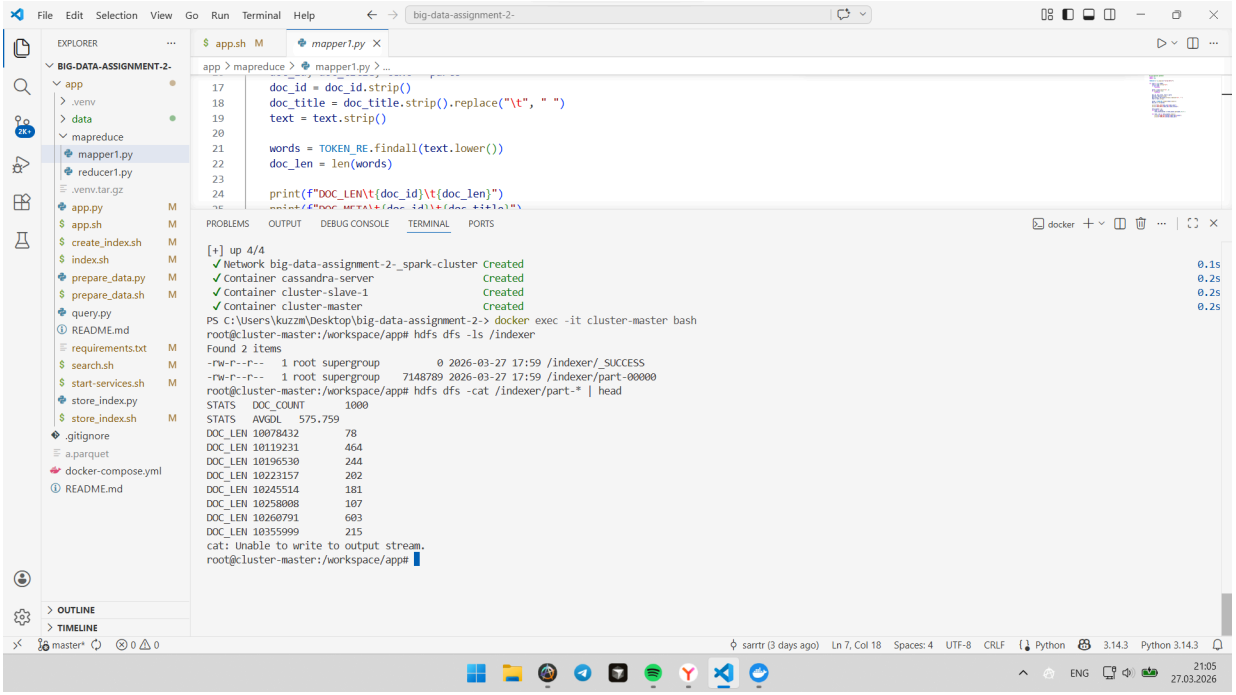
It checks whether a query argument was provided, preventing the Spark job from starting with an empty query. Then it determines the absolute path of the app directory using SCRIPT_DIR. After that it launches query.py on YARN in client mode. The options spark.yarn.appMasterEnv.PYSPARK_PYTHON and spark.executorEnv.PYSPARK_PYTHON point both the application master and executors to ./.venv/bin/python. Finally, the query text is passed to query.py as command-line arguments.

Demonstration

How to run:

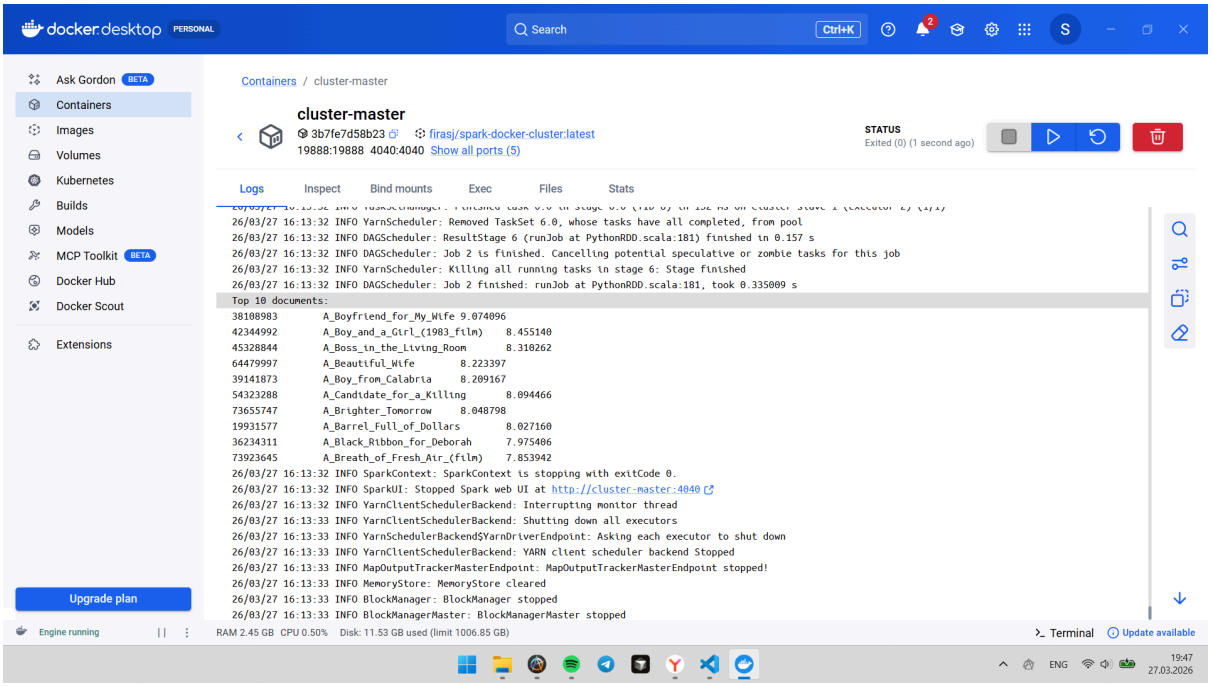
1. clone the repository with ‘git clone https://github.com/sarrtr/big-data-assignment-2-.git’
2. put a.parquet file to the root of the project
3. from the root of the project run ‘docker compose up’
4. change the query if you want in app.sh
5. find the result of the query starting with ‘Top 10 documents:’

Indexing:

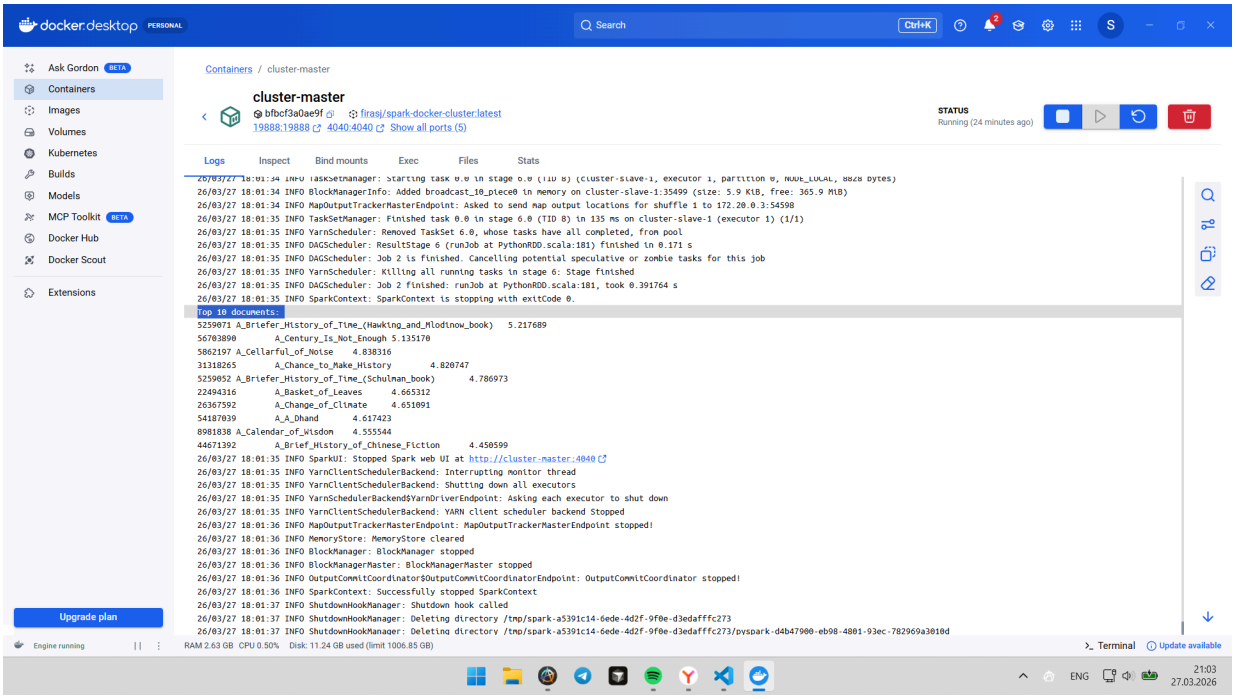


Queries with explanations:

Q1: “popular Italian films”



Q2: “English books”



The implemented search engine simply retrieves documents based on the presence and frequency of query terms in indexed text rather than by true semantic meaning. So, sometimes the query result appears incorrect.

For example, for query “popular Italian films” the best result is “A boyfriend for my wife” which is an Argentine romantic comedy film. The word “Italian” occurs

in context of language of one of the original film remakes, not in the meaning of the Italian film the text is about. The remaining 9 results are relevant.

Another example: for query “English books” one of the results is “A A Dhand” - British-Asian crime-writer, it is not a book, but the author that writes books in English.

This shows a limitation of term-based search: the engine does not understand context and semantic relevance.